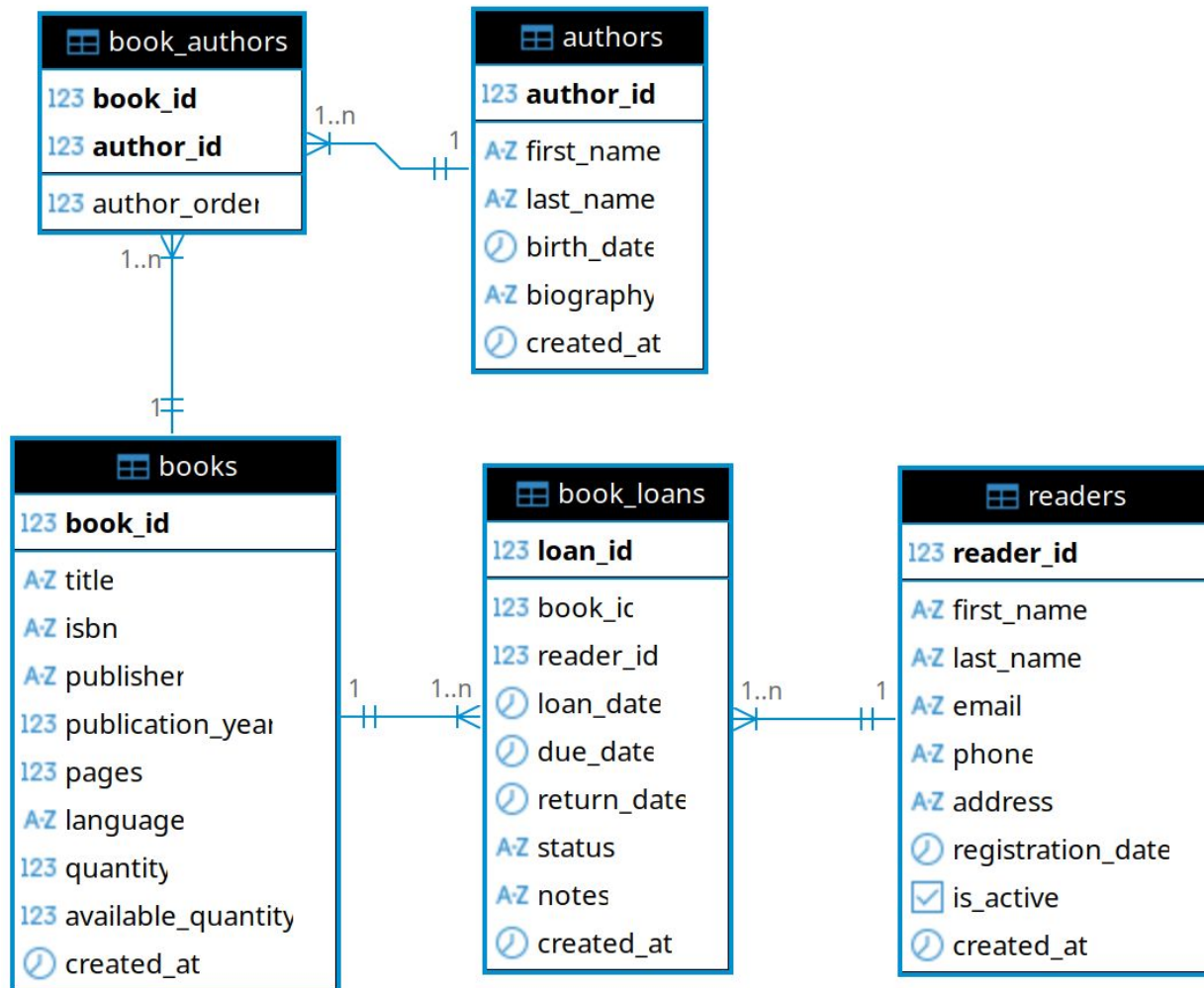


# Группировка и агрегатные функции. Подзапросы.

Лекция 6



# Группировка и агрегатные функции

# Агрегатные функции

Набор значений -> единственный результат

- count – количество строк
- min / max – минимальное / максимальное значение
- sum – сумма значений
- avg – среднее значение
- any\_value – произвольное не NULL значение
- bool\_and – возвращает true, если все значения не NULL
- bool\_or – возвращает true, если хотя бы одно значение не NULL

<https://postgrespro.ru/docs/postgresql/current/functions-aggregate#FUNCTIONS-AGGREGATE-TABLE>

# Агрегатные функции

СКОЛЬКО КНИГ: `SELECT COUNT(*) AS total_books FROM books;`

Статистика по книгам:

```
SELECT
    COUNT(*) AS total_books,
    MIN(pages) AS min_pages,
    MAX(pages) AS max_pages,
    ROUND(AVG(pages)) AS avg_pages,
    SUM(pages) AS total_pages
FROM books;
```

|   | <input type="checkbox"/> total_books ▾ | ÷ | <input type="checkbox"/> min_pages ▾ | ÷ | <input type="checkbox"/> max_pages ▾ | ÷ | <input type="checkbox"/> avg_pages ▾ | ÷ | <input type="checkbox"/> total_pages ▾ | ÷ |
|---|----------------------------------------|---|--------------------------------------|---|--------------------------------------|---|--------------------------------------|---|----------------------------------------|---|
| 1 | 27                                     |   | 128                                  |   | 1300                                 |   | 463                                  |   | 12499                                  |   |

# Группировка

GROUP BY – группирует строки по одинаковым значениям указанных столбцов и применяет агрегатные функции к каждой группе. Строки таблицы заменяются на сгруппированные строки.

```
SELECT publisher, COUNT(*) AS book_count  
FROM books GROUP BY publisher ORDER BY book_count DESC;
```

|   | publisher | book_count |
|---|-----------|------------|
| 1 | АСТ       | 11         |
| 2 | Азбука    | 10         |
| 3 | Махаон    | 3          |
| 4 | Эксмо     | 3          |

Группировка без агрегатных функций выводит набор различающихся значений, что эквивалентно запросу с DISTINCT без группировки

# Группировка по нескольким полям

```
SELECT
    publisher,
    publication_year,
    COUNT(*) AS book_count
FROM books
GROUP BY publisher, publication_year
ORDER BY publisher, publication_year DESC;
```

|   | publisher | publication_year | book_count |
|---|-----------|------------------|------------|
| 1 | Азбука    | 2021             | 1          |
| 2 | Азбука    | 2020             | 6          |
| 3 | Азбука    | 2019             | 3          |
| 4 | АСТ       | 2021             | 4          |
| 5 | АСТ       | 2020             | 3          |
| 6 | АСТ       | 2019             | 3          |
| 7 | АСТ       | 2018             | 1          |

# Группировка

```
SELECT
```

```
    publisher,  
    COUNT(*) AS book_count,  
    SUM(quantity) AS total_copies,  
    CEIL(AVG(pages)) AS avg_pages,  
    MIN(publication_year) AS oldest_year,  
    MAX(publication_year) AS newest_year
```

```
FROM books
```

```
GROUP BY publisher
```

```
ORDER BY book_count DESC;
```

Все столбцы SELECT должны быть  
либо в GROUP BY, либо  
результатами агрегатных функций

|   | publisher | book_count | total_copies | avg_pages | oldest_year | newest_year |
|---|-----------|------------|--------------|-----------|-------------|-------------|
| 1 | АСТ       | 11         | 36           | 491       | 2018        | 2021        |
| 2 | Азбука    | 10         | 28           | 349       | 2019        | 2021        |
| 3 | Махаон    | 3          | 19           | 475       | 2021        | 2021        |
| 4 | Эксмо     | 3          | 9            | 731       | 2018        | 2020        |



# Группировка с фильтрацией

```
SELECT publisher, COUNT(*) AS book_count  
FROM books GROUP BY publisher  
HAVING COUNT(*) > 5 ORDER BY book_count DESC;
```

|   | publisher | book_count |
|---|-----------|------------|
| 1 | АСТ       | 11         |
| 2 | Азбука    | 10         |

HAVING фильтрует уже сгруппированные строки

# Что делает этот запрос?

```
SELECT
```

```
    r.first_name || ' ' || r.last_name AS reader_name,  
    COUNT(b1.loan_id) AS loan_count
```

```
FROM readers r
```

```
JOIN book_loans b1 ON r.reader_id = b1.reader_id
```

```
GROUP BY r.reader_id, r.first_name, r.last_name
```

```
HAVING COUNT(b1.loan_id) > 1
```

```
ORDER BY loan_count DESC;
```

# Что делает этот запрос?

SELECT

```
r.first_name || ' ' || r.last_name AS reader_name,  
COUNT(b1.loan_id) AS loan_count
```

FROM readers r

JOIN book\_loans b1 ON r.reader\_id = b1.reader\_id

GROUP BY r.reader\_id, r.first\_name, r.last\_name

HAVING COUNT(b1.loan\_id) > 1

ORDER BY loan\_count DESC;

|   | ☐ reader_name 🔽 | ÷ ☐ loan_count 🔽 | ÷ |
|---|-----------------|------------------|---|
| 1 | Елена Кузнецова |                  | 2 |
| 2 | Анна Соколова   |                  | 2 |
| 3 | Мария Петрова   |                  | 2 |
| 4 | Сергей Морозов  |                  | 2 |
| 5 | Алексей Смирнов |                  | 2 |
| 6 | Иван Иванов     |                  | 2 |

Находит читателей, которые  
брали более 1 книги

# Порядок выполнения запроса SELECT

1. Вычисляются все элементы после FROM: реальные и виртуальные таблицы, соединения
2. Применяется WHERE: из набора исключается ненужное
3. Применяется GROUP BY: набор трансформируется в соответствии с атрибутами группировки, применяются агрегатные функции
4. Применяется HAVING: исключаются ненужные сгруппированные строки
5. Вычисляются выражения внутри SELECT: вызываются функции, применяется DISTINCT
6. Применяется ORDER BY
7. Применяется LIMIT и OFFSET

```
SELECT DISTINCT
    r.reader_id,
    UPPER(r.last_name || ' ' || r.first_name) AS reader_name, -- вычисления
    COUNT(bl.loan_id) AS total_loans,
    COUNT(CASE WHEN bl.status = 'returned' THEN 1 END) AS returned_count
FROM readers r -- 1. FROM
INNER JOIN book_loans bl ON r.reader_id = bl.reader_id -- JOIN

WHERE    bl.loan_date >= '2020-01-01' -- 2. WHERE
AND r.is_active = true -- фильтрация строк

GROUP BY r.reader_id, r.last_name, r.first_name -- 3. GROUP BY

HAVING COUNT(CASE WHEN bl.status = 'returned' THEN 1 END) >= 1 -- 4. HAVING
AND COUNT(bl.loan_id) >= 1 -- фильтрация групп

ORDER BY total_loans DESC -- 6. ORDER BY

LIMIT 5 OFFSET 0; -- 7. LIMIT OFFSET
```

# Оконные функции

# Оконные функции

Что если хотим агрегатные функции, но не хотим терять строки с данными?

```
SELECT
    title,
    publisher,
    pages,
    COUNT(*) OVER() AS total_books -- оконная функция
FROM books
ORDER BY publisher, title;
```

|   | 📖 title 📄                | ÷ | 📖 publisher 📄 | ÷ | 📖 pages 📄 | ÷ | 📖 total_books 📄 | ÷ |
|---|--------------------------|---|---------------|---|-----------|---|-----------------|---|
| 1 | 451 градус по Фаренгейту |   | Азбука        |   | 256       |   | 27              |   |
| 2 | Вишнёвый сад             |   | Азбука        |   | 256       |   | 27              |   |
| 3 | Дама с собачкой          |   | Азбука        |   | 320       |   | 27              |   |
| 4 | Доктор Живаго            |   | Азбука        |   | 704       |   | 27              |   |
| 5 | Белый Флот               |   | Азбука        |   | 111       |   | 27              |   |

# Оконные функции: разделы (partitions)

Мы можем сгруппировать выборку по разделам и применять агрегатные функции в рамках раздела

```
SELECT title, publisher, pages,  
       COUNT(*) OVER(PARTITION BY publisher) AS books_in_publisher,  
       FLOOR(AVG(pages) OVER(PARTITION BY publisher)) AS avg_pages_publisher  
FROM books  
ORDER BY publisher, title;
```

|    | title ▾             | publisher ▾ | pages ▾ | books_in_publisher ▾ | avg_pages_publisher ▾ |
|----|---------------------|-------------|---------|----------------------|-----------------------|
| 7  | Марсианские хроники | Азбука      | 352     | 10                   | 348                   |
| 8  | Мёртвые души        | Азбука      | 352     | 10                   | 348                   |
| 9  | Ревизор             | Азбука      | 160     | 10                   | 348                   |
| 10 | Сто лет одиночества | Азбука      | 448     | 10                   | 348                   |
| 11 | 1984                | АСТ         | 320     | 11                   | 490                   |
| 12 | Анна Каренина       | АСТ         | 864     | 11                   | 490                   |
| 13 | ...                 | ...         | ...     | ...                  | ...                   |



# Оконные функции: оконный кадр (window frame)

ORDER BY внутри OVER – определяет понятие “оконного кадра”.  
Оконный кадр – набор строк с первой по текущую внутри раздела

```
SELECT title, publisher, publication_year, pages,  
       -- Накопительная сумма страниц по издательству (по годам)  
       SUM(pages) OVER (  
         PARTITION BY publisher  
         ORDER BY publication_year  
       ) AS cumulative_pages  
FROM books  
ORDER BY publisher, publication_year;
```

| title                    | publisher | publication_year | pages | cumulative_pages |
|--------------------------|-----------|------------------|-------|------------------|
| Братья Карамазовы        | Эксмо     | 2018             | 880   | 880              |
| Преступление и наказание | Эксмо     | 2019             | 671   | 1551             |
| Идиот                    | Эксмо     | 2020             | 640   | 2191             |

# Оконные функции: оконный кадр (window frame)

Все строки с одним значением критерия ORDER BY внутри раздела, включаются в оконный кадр

| title ▾                  | publisher ▾ | p ▾  | pages ▾ | cum... ▾ |
|--------------------------|-------------|------|---------|----------|
| Дама с собачкой          | Азбука      | 2019 | 320     | 928      |
| Мёртвые души             | Азбука      | 2019 | 352     | 928      |
| 451 градус по Фаренгейту | Азбука      | 2019 | 256     | 928      |
| Вишнёвый сад             | Азбука      | 2020 | 256     | 3072     |
| Капитанская дочка        | Азбука      | 2020 | 224     | 3072     |
| Сто лет одиночества      | Азбука      | 2020 | 448     | 3072     |
| Доктор Живаго            | Азбука      | 2020 | 704     | 3072     |
| Марсианские хроники      | Азбука      | 2020 | 352     | 3072     |
| Ревизор                  | Азбука      | 2020 | 160     | 3072     |
| Евгений Онегин           | Азбука      | 2021 | 416     | 3488     |
| Архипелаг ГУЛАГ          | АСТ         | 2018 | 608     | 608      |
| Прощай, оружие!          | АСТ         | 2019 | 352     | 2176     |
| Белая гвардия            | АСТ         | 2019 | 352     | 2176     |
| Анна Каренина            | АСТ         | 2019 | 864     | 2176     |

# Оконные функции: функции ранжирования

```
SELECT title, publisher, pages,  
       -- Номер строки внутри издательства (по убыванию страниц)  
       ROW_NUMBER() OVER(PARTITION BY publisher ORDER BY pages DESC) AS row_num,  
       -- Ранг (одинаковые значения = одинаковый ранг, пропуск номеров)  
       RANK() OVER(PARTITION BY publisher ORDER BY pages DESC) AS rank,  
       -- Плотный ранг (без пропусков)  
       DENSE_RANK() OVER(PARTITION BY publisher ORDER BY pages DESC) AS d_rank  
FROM books ORDER BY publisher, pages DESC;
```

| title               | publisher | pages | row_num | rank | d_rank |
|---------------------|-----------|-------|---------|------|--------|
| Доктор Живаго       | Азбука    | 704   | 1       | 1    | 1      |
| Сто лет одиночества | Азбука    | 448   | 2       | 2    | 2      |
| Евгений Онегин      | Азбука    | 416   | 3       | 3    | 3      |
| Марсианские хроники | Азбука    | 352   | 4       | 4    | 4      |
| Мёртвые души        | Азбука    | 352   | 5       | 4    | 4      |
| Дама с собачкой     | Азбука    | 320   | 6       | 6    | 5      |

# Оконные функции: именованные окна

```
SELECT title, publisher, pages,  
       ROW_NUMBER() OVER by_publisher_and_pages AS row_num,  
       RANK() OVER by_publisher_and_pages AS rank,  
       DENSE_RANK() OVER by_publisher_and_pages AS d_rank  
FROM books  
  
-- Определяем окно один раз  
WINDOW by_publisher_and_pages AS (  
    PARTITION BY publisher ORDER BY pages DESC  
)  
ORDER BY publisher, pages DESC;
```

# Оконные функции: изменение оконного кадра

```
SELECT title, publication_year as year, pages,  
       ROUND(AVG(pages) OVER(  
         ORDER BY publication_year, pages  
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- ← определяем окно  
       )) AS moving_avg_3  
FROM books ORDER BY publication_year, pages;
```

| title                    | year | pages | moving_avg_3 |
|--------------------------|------|-------|--------------|
| Одиссея капитана Блада   | 2018 | 608   | 608          |
| Братья Карамазовы        | 2018 | 880   | 744          |
| 451 градус по Фаренгейту | 2019 | 256   | 581          |
| Дама с собачкой          | 2019 | 320   | 485          |
| Прощай, оружие!          | 2019 | 352   | 309          |
| Белая гвардия            | 2019 | 352   | 341          |
| Мёртвые души             | 2019 | 352   | 352          |
| Преступление и наказание | 2019 | 671   | 458          |

# Подзапросы

# Подзапросы в WHERE

Найдем книги толще среднего:

```
SELECT title, pages FROM books
WHERE pages > (
    SELECT AVG(pages) FROM books -- скалярный подзапрос
) ORDER BY pages DESC;
```

Или найдем читателей, которые брали книги:

```
SELECT first_name, last_name FROM readers
WHERE reader_id IN (
    SELECT DISTINCT reader_id FROM book_loans -- подзапрос в предикате IN
) ORDER BY last_name;
```

# Коррелированные подзапросы

Как найти самую толстую книгу в каждом издательстве?

```
SELECT
    b1.title,
    b1.publisher,
    b1.pages
FROM books b1
WHERE b1.pages = (
    SELECT MAX(b2.pages)
    FROM books b2
    WHERE b2.publisher = b1.publisher -- ← подзапрос зависит от внешнего
)
ORDER BY b1.publisher;
```

А как найти топ-3 толстых книг по каждому издательству?



# Ключевое слово EXISTS

Найдем издательства, у которых есть книги больше 500 страниц:

```
SELECT DISTINCT publisher FROM books b1
WHERE EXISTS (
    SELECT 1
    FROM books b2 WHERE b2.publisher = b1.publisher
    AND b2.pages > 500
) ORDER BY publisher;
```

Или найдем авторов, книг которых нет в библиотеке:

```
SELECT first_name, last_name FROM authors a
WHERE NOT EXISTS (
    SELECT 1
    FROM book_authors ba WHERE ba.author_id = a.author_id
) ORDER BY last_name;
```

# Подзапросы в SELECT

```
SELECT title, publisher, pages,  
       (SELECT COUNT(*) FROM books) AS total_books, - скалярные подзапросы  
       (SELECT AVG(pages) FROM books) AS avg_pages_all  
FROM books ORDER BY pages DESC;
```

Для каждой книги, покажем количество выдач:

```
SELECT b.title, b.publisher,  
       (  
         SELECT COUNT(*)           -- коррелированный подзапрос  
         FROM book_loans bl  
         WHERE bl.book_id = b.book_id  
       ) AS times_borrowed  
FROM books b ORDER BY times_borrowed DESC;
```

# Подзапрос в FROM

Выведем топ читателей по количеству выдач

```
SELECT
    reader_name,
    loan_count,
    CASE
        WHEN loan_count >= 3 THEN '★ Активный'
        ELSE 'Обычный'
    END AS status
FROM (
    SELECT
        r.first_name || ' ' || r.last_name AS reader_name,
        COUNT(bl.loan_id) AS loan_count
    FROM readers r
    JOIN book_loans bl ON r.reader_id = bl.reader_id
    GROUP BY r.reader_id
) AS reader_stats
ORDER BY loan_count DESC;
```

# Подзапрос в FROM

Найдем топ-3 по толщине книг по каждому издательству!

```
SELECT *
FROM (
    SELECT
        publisher,
        title,
        pages,
        ROW_NUMBER() OVER(PARTITION BY publisher ORDER BY pages DESC) AS
rank_num
    FROM books
) ranked
WHERE rank_num <= 3
ORDER BY publisher, rank_num;
```

# Как работает такой запрос?

```
SELECT
    title,
    pages,
    pages - avg_pages AS diff_from_avg
FROM books, (
    SELECT AVG(pages) AS avg_pages
    FROM books
) AS avg_data
WHERE pages > avg_pages
ORDER BY diff_from_avg DESC;
```

# Как работает такой запрос?

```
SELECT
    title,
    pages,
    pages - avg_pages AS diff_from_avg
FROM books, (
    -- CROSS JOIN books и вирт. таблицы с одной строкой
    SELECT AVG(pages) AS avg_pages -- скалярный подзапрос (одна строка)
    FROM books
) AS avg_data
WHERE pages > avg_pages
ORDER BY diff_from_avg DESC;
```

# Общие табличные выражения (common table expressions)(CTE)

# Проблема: повторяющиеся запросы

```
SELECT title, pages,  
       pages - (SELECT AVG(pages) FROM books) AS diff_from_avg,  
FROM books  
WHERE pages > (SELECT AVG(pages) FROM books)  
ORDER BY pages DESC;
```

Решение: вынесем запрос отдельно

```
WITH avg_stats AS (    -- общее табличное выражение  
    SELECT AVG(pages) AS avg_pages  
    FROM books  
)  
SELECT title, pages,  
       pages - avg_pages AS diff_from_avg  
FROM books, avg_stats  
WHERE pages > avg_pages  
ORDER BY diff_from_avg DESC;
```



# Несколько СТЕ

WITH

book\_stats AS ( -- Первое СТЕ: статистика по книгам

SELECT

COUNT(\*) AS total\_books,

AVG(pages) AS avg\_pages

FROM books

), -- Второе СТЕ: статистика по читателям

reader\_stats AS (

SELECT

COUNT(\*) AS total\_readers,

COUNT(\*) FILTER (WHERE is\_active = true) AS active\_readers

FROM readers

)

SELECT \* FROM book\_stats, reader\_stats;

| <input type="checkbox"/> total_books ▾ | ÷ <input type="checkbox"/> avg_pages ▾ | ÷ <input type="checkbox"/> total_readers ▾ | ÷ <input type="checkbox"/> active_readers ▾ | ÷ |
|----------------------------------------|----------------------------------------|--------------------------------------------|---------------------------------------------|---|
| 27                                     | 462.9259259259259259                   | 15                                         | 15                                          |   |

# Цепочки CTE

WITH

loan\_counts AS ( -- Первое CTE: статистика по читателям

SELECT reader\_id, COUNT(\*) AS cnt

FROM book\_loans

GROUP BY reader\_id

),

avg\_count AS ( -- Второе CTE: скалярный подзапрос, использует первый CTE

SELECT AVG(cnt) AS avg FROM loan\_counts

)

SELECT \*

FROM loan\_counts, avg\_count

WHERE cnt > avg;

# Рекурсивные CTE

Выведем числа от 1 до 5

```
WITH RECURSIVE numbers AS (  
    -- Базовый случай (anchor)  
    SELECT 1 AS n  
  
    UNION ALL  
  
    -- Рекурсивная часть  
    SELECT n + 1  
    FROM numbers  
    WHERE n < 5  
)  
SELECT n FROM numbers;
```

| n |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |

# Рекурсивные СТЕ: практическое применение

```
WITH RECURSIVE date_series AS ( -- Генерация дат для отчета (последние 7 дней)
    SELECT CURRENT_DATE - 6 AS report_date

    UNION ALL

    SELECT report_date + 1
    FROM date_series
    WHERE report_date < CURRENT_DATE
)
SELECT
    ds.report_date,
    TO_CHAR(ds.report_date, 'Day') AS day_name,
    COUNT(bl.loan_id) AS loans_count
FROM date_series ds
LEFT JOIN book_loans bl ON DATE(bl.loan_date) = ds.report_date
GROUP BY ds.report_date ORDER BY ds.report_date;
```

# Рекурсивные СТЕ: практическое применение

*-- Иерархия (если бы была структура категорий)*

```
CREATE TEMP TABLE book_categories (  
    category_id INT,  
    category_name text,  
    parent_id INT  
);
```

*-- Пример структуры: Художественная литература -> Русская классика -> Толстой*

```
INSERT INTO book_categories VALUES  
(1, 'Все книги', NULL),  
(2, 'Художественная литература', 1),  
(3, 'Научная литература', 1),  
(4, 'Русская классика', 2),  
(5, 'Зарубежная классика', 2),  
(6, 'Толстой', 4),  
(7, 'Достоевский', 4);
```

# Рекурсивные СТЕ: визуализация иерархии категорий

```
WITH RECURSIVE category_tree AS (  
    SELECT      -- Корневая категория  
        category_id, category_name, parent_id,  
        1 AS level, category_name AS path  
    FROM book_categories  
    WHERE parent_id IS NULL  
  
    UNION ALL  
  
    SELECT -- Дочерние категории  
        bc.category_id, bc.category_name, bc.parent_id,  
        ct.level + 1,  
        ct.path || ' > ' || bc.category_name  
    FROM book_categories bc  
    JOIN category_tree ct ON bc.parent_id = ct.category_id  
)  
SELECT REPEAT('  ', level - 1) || category_name AS hierarchy,  
       level, path  
FROM category_tree ORDER BY path;
```

# Рекурсивные СТЕ: визуализация иерархии категорий

| <div>hierarchy</div>      | <div>level</div> | <div>path</div>                                                        |
|---------------------------|------------------|------------------------------------------------------------------------|
| Все книги                 | 1                | Все книги                                                              |
| Научная литература        | 2                | Все книги > Научная литература                                         |
| Художественная литература | 2                | Все книги > Художественная литература                                  |
| Зарубежная классика       | 3                | Все книги > Художественная литература > Зарубежная классика            |
| Русская классика          | 3                | Все книги > Художественная литература > Русская классика               |
| Достоевский               | 4                | Все книги > Художественная литература > Русская классика > Достоевский |
| Толстой                   | 4                | Все книги > Художественная литература > Русская классика > Толстой     |

# Представления (views)



# Зачем нужны представления?

Проблема: постоянно используем один и тот же запрос (подзапрос)

Почему бы нам не сохранить его в СУБД?

```
CREATE VIEW available_books AS
SELECT
    book_id, title, publisher, publication_year, pages, available_quantity
FROM books WHERE available_quantity > 0;
```

Теперь можем использовать его как таблицу

```
SELECT * FROM available_books;
```

# Изменение представлений

```
CREATE OR REPLACE VIEW available_books AS
SELECT
    book_id,
    title,
    publisher,
    publication_year,
    pages,
    available_quantity,
    quantity,
    ROUND(available_quantity * 100.0 / quantity, 1) AS percent -- добавили
FROM books WHERE available_quantity > 0;
```

# Изменение представлений

А вот так не сработает:

```
CREATE OR REPLACE VIEW available_books AS
SELECT
    book_id as id, -- нельзя переименовывать или удалять поля
    title,
    publisher,
    publication_year,
    pages,
    available_quantity,
    quantity
FROM books WHERE available_quantity > 0;
```

```
[42P16] ERROR: cannot change name of view column "book_id" to "id"
Hint: Use ALTER VIEW ... RENAME COLUMN ... to change name of view column instead.
```

# Изменение представлений

Всегда можно удалить и создать заново:

```
DROP VIEW available_books;
```

```
CREATE OR REPLACE VIEW available_books AS  
SELECT
```

```
    book_id as id,  
    title,  
    publisher,  
    publication_year,  
    pages,  
    available_quantity,  
    quantity
```

```
FROM books
```

```
WHERE available_quantity > 0;
```